



ທະນາຄານຮ່ວມພັດທະນາ
Joint Development Bank

JDB BACKEND WEB API V1.2

Payment link Spec



NOVEMBER 28, 2022

JOINT DEVELOPMENT BANK
82 LANEXANG ROAD, VIENTIANE LAO P.D.R

Table Content

1.	Overview.....	2
1.1.	Request Header.....	2
1.2.	Response Success Format	2
1.3.	Response ERROR Format	2
1.4.	Process flow	3
2.	Functions of API	4
2.1.	Authentication	4
2.1.1.	Request Body.....	4
2.1.2.	Response Data	4
2.2.	Generate emvCode	5
2.2.1.	Request Body.....	5
2.2.2.	Response Data	6
2.3.	Check transaction status.....	6
2.3.1.	Request Body.....	6
2.3.2.	Response Data	7
2.4.	Example code for algorithm encrypt data to SignedHash key.....	8
3.	Subscription transaction (PubNub)	9
3.1.	Introduction.....	9
3.2.	Initialize PubNub.....	9
3.3.	Channel name	9
3.4.	Subscribe to Channels.....	9
3.5.	Receive Message.....	10
4.	Contact information	11



JDB Service

Payment link spec v.1.2

1. Overview

This document is designed to provide standard for partner, this is API platform for HTTPs protocol interface works as allow partner can access and partner should be understanding QR payment process.

1.1. Request Header

Content-Type : << Application/json >>

Authorization : << Bearer Token on authentication service >>

SignedHash : << Hash data Key >>

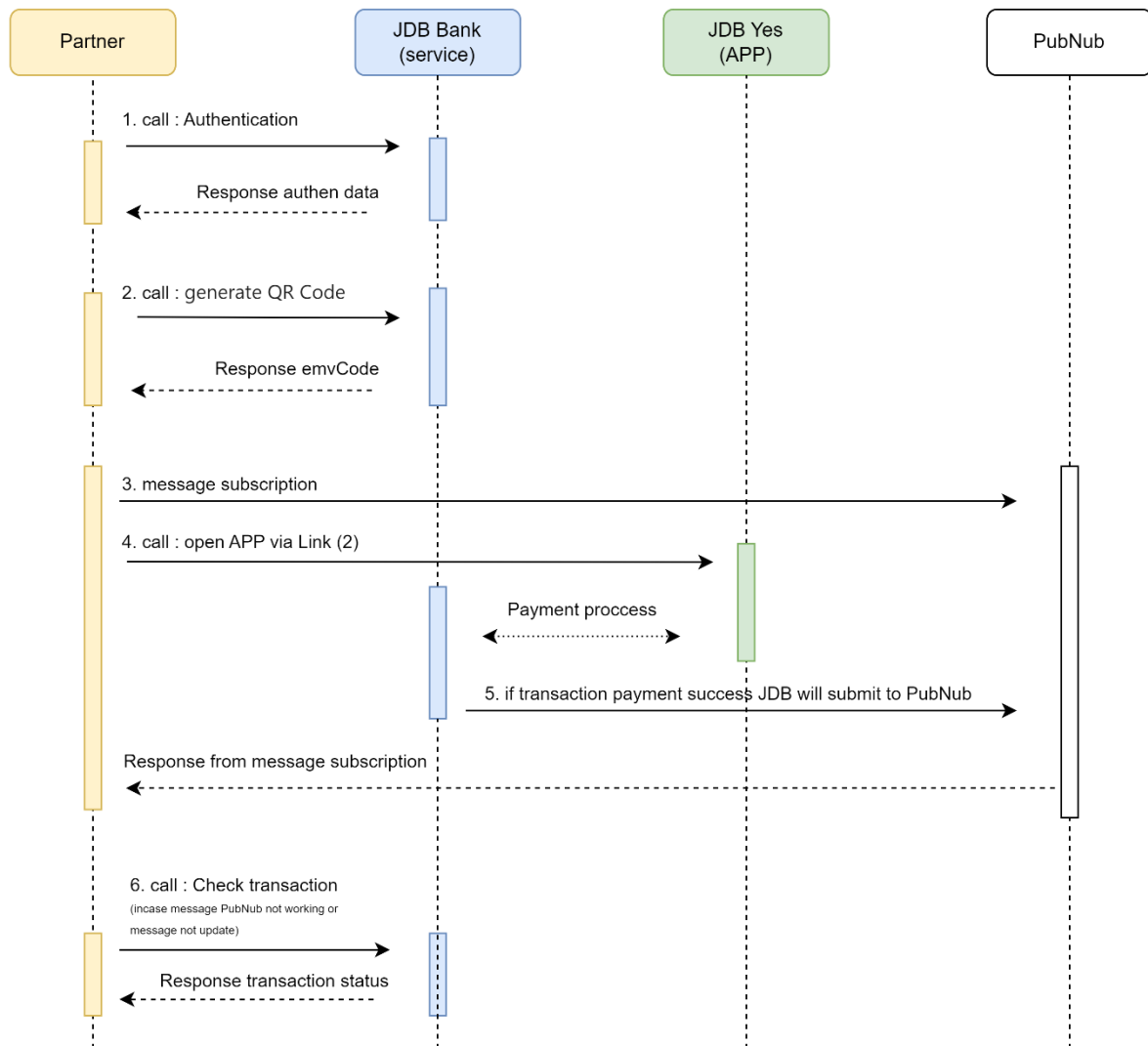
1.2. Response Success Format

Field	Type	Description
timestamp	String	e.g: 2021-05-21 09:12:24.595
success	Boolean	True
message	String	
transactionId	String	requestId from message request
status	String	
data	Array Object	

1.3. Response ERROR Format

Field	Type	Description
timestamp	String	e.g: 2021-05-21 09:12:24.595
success	Boolean	False
status	String	
message	String	
data	Array Object	

1.4. Process flow



2.2. Generate emvCode

URL>> https://(ip address or domain name):(port)/api/uat/dynamic/generateQr

Method >> POST

2.2.1. Request Body

Field	Type	Description	Mandatory
requestId	String (up to 25)	Generate with unique ID for each transaction	Y
partnerId	String (up to 10)	JDB will provide	Y
merchantId	String (25)	JDB will provide	Y
txnAmount	Number	Transaction amount	Y
billNumber	String (up to 25)	Bill number	Y
terminalId	String (up to 25)	Reference label	Y
terminalLabel	String (up to 25)	Terminal label	Y
mobileNo	String (up to 25)	Mobile number	Y

Remark:

Format APP link UAT: https://jdbyes.laoit.dev/yespay/<emv>

Example

```
{
  "requestId": "2022110314567895",
  "partnerId": "NCC",
  "merchantId": "xxxx",
  "txnAmount": 20000,
  "billNumber": "JDB547885467",
  "terminalId": "121312121",
  "terminalLabel": "JDB000006",
  "mobileNo": "2077004545"
}
```

2.2.2. Response Data

Field	Type	Description	Mandatory
mcid	String (25)		Y
emv	string		Y

Example

```
{
  "timestamp": "2022-11-03 13:56:21.806",
  "success": true,
  "message": "SUCCESS",
  "transactionId": "2022110314567895",
  "status": "OK",
  "data": {
    "mcid": "xxxxxxxxxxxxxx",
    "emv":
"00020101021238680016A005266284662577010832170418020300203259YBH6W1OMFDJ1EIQMU6XP5X5U520
4525153034185405200005802LA5910LaoLottery6009Vientiane62660112JDB547885468020100301004010050100
603NCC070912131212108010090106304A8D6"
  }
}
```

Noted: please add format apblink for access to open JDB Yes app via process flow at step 4.

Example:

<https://jdbyes.laoit.dev/yespay/00020101021238680016A00526628466257701083217041802030020325Q36BHDB5FYSCCWA9OVEPN792W5204525153034185405200005802LA5910LaoLottery6009Vientiane626701125201547885460210207700454505091213121210603NCC0709JDB00000408006304879B>

2.3. Check transaction status

URL>> [https://\(ip address or domain name\):\(port\)/api/uat/dynamic/checkTransaction](https://(ip address or domain name):(port)/api/uat/dynamic/checkTransaction)

Method >> POST

2.3.1. Request Body

Field	Type	Description	Mandatory
requestId	String (up to 25)	Generate by partner for each transaction	Y
billNumber	String (up to 25)	Bill number	Y

Example

```
{
  "requestId": "54545564654655",
  "billNumber": "1201547885411456"
}
```

2.3.2. Response Data

Field	Type	Description	Mandatory
message	String (up to 25)		Y
refNo	String (up to 25)	Bank reference	Y
exReferenceNo	String (up to 25)	Bank external reference	Y
merchantName	String (up to 25)	Merchant name	Y
memo	String (up to 250)	Transaction's bank description	Y
txnDateTime	String (up to 25)	Transaction date and time	Y
txnAmount	Number	Transaction amount	Y
billNumber	String (up to 25)	Bill number	Y
sourceAccount	String (up to 25)	Customer account number	Y
sourceName	String (up to 25)	Customer name	Y
sourceCurrency	String (up to 25)	Customer account currency	Y

Example

```
{
  "timestamp": "2022-10-28 09:09:47.908",
  "success": true,
  "message": "SUCCESS",
  "transactionId": "54545564654655",
  "status": "OK",
  "data": {
    "message": "SUCCESS",
    "refNo": "001MBQR222260588",
    "exReferenceNo": "WGMVWWJXoXLGQ6IVZ",
    "merchantName": "PAYMENT TEST UAT",
    "memo": "Yes Pay: From|NONG FAR |To| PAYMENT TEST UAT|hhhh |",
    "txnDateTime": "2022-10-27 12:15:18",
    "txnAmount": 20000,
    "billNumber": "520154788546",
    "sourceAccount": "00120010010062660",
    "sourceName": "NONG FAR",
    "sourceCurrency": "LAK"
  }
}
```


2.4. Example code for algorithm encrypt data to SignedHash key

```

public String generateHash(String originalString) throws JsonProcessingException {
    SecretKeySpec secretKeySpec = new SecretKeySpec(SECRET_KEY.getBytes(), "HmacSHA256");
    Mac mac; // originalString=jsonMapper.writeValueAsString(originalString);
    try {
        mac = Mac.getInstance(HMAC_SHA256);
        mac.init(secretKeySpec);
        byte[] encodedhash = mac.doFinal(originalString.getBytes(StandardCharsets.UTF_8));

        return bytesToHex(encodedhash);

    } catch (NoSuchAlgorithmException e) {
        e.printStackTrace();
    } catch (InvalidKeyException e) {
        e.printStackTrace();
    }
    return null;
}

public static String bytesToHex(byte[] hash) {
    StringBuffer hexString = new StringBuffer();
    for (int i = 0; i < hash.length; i++) {
        String hex = Integer.toHexString(0xff & hash[i]);
        if (hex.length() == 1)
            hexString.append('0');
        hexString.append(hex);
    }
    return hexString.toString();
}

```

3. Subscription transaction (PubNub)

3.1. Introduction

PubNub is reliable. It's efficient. It's fast. It's everything we were looking for.

<https://www.pubnub.com/docs>

3.2. Initialize PubNub

Before you start calling PubNub APIs, you must initialize the PubNub object in your app. This initialization uses the publish and subscribe keys you got from JDB provided and allows you to work with PubNub functionality.

more information <https://www.pubnub.com/docs/basics/initialize-pubnub>

Example: for JAVA

```
final UserId userId = new UserId("UserID");
pnConfiguration = new PNConfiguration(userId);
pnConfiguration.setSubscribeKey("SubscribeKey");
PubNub pubnub = new PubNub(pnConfiguration);
```

Example: for JavaScript

```
var pubnub = new PubNub({
  subscribeKey: "SubscribeKey ",
  uuid: "UserID ",
});
```

Note:

- "SubscribeKey": is Subscribe key provided from JDB
- "UserID": A UUID (Universally Unique Identifier) is a unique string of up to 64 characters used to identify a client

3.3. Channel name

A **channel** is a mechanism through which the data is transmitted from one device to another within the PubNub Real-time Communication Platform

A channel name can be any alphanumeric string up to 92 UTF-8 characters. Channel names are unique per PubNub key set.

ChannelName format is MerchantId+"_" +billNumber

➤ Invalid characters

, : * / and Unicode Zero, whitespace, and non-printable ASCII characters.

➤ Valid characters

- = @ . ! \$ # % & ;

3.4. Subscribe to Channels

For more information: <https://www.pubnub.com/docs/channels/subscribe>

Example: JavaScript

```
pubnub.subscribe({
  channels: [channelName],
});
```

Example: Java

```
pubnub.subscribe()
    .channels(Arrays.asList(channelName))
    .execute();
```

Note: "channelName" The partner can use MerchantId to subscription, format is MerchantId+"_"+billNumber

3.5. Receive Message

for receive a message, your client must implement a listener and subscribe to a channel in which the message is being published. <https://www.pubnub.com/docs/messages/receive>

Example: JavaScript

```
pubnub.addListener({
  message: function (msg) {
    console.log(msg.message);
  },
});
```

Example: JAVA

```
pubnub.addListener(new SubscribeCallback() {
    @Override
    public void message(@NotNull PubNub pubnub, @NotNull
    PNMessageResult pnMessageResult) {
        System.out.println("Message: " +
            pnMessageResult.getMessage());
    }
})
```

Example Code for JavaScript

```
<script>
  var pubnub = new PubNub({
    subscribeKey: "sub-c-3014e386-6988-47bb-9fe0-2c5d7a61544c",
    uuid: "6e576d4c-47df-4469-8455-36af98e8d8f5",
  });
  var channel = "channelName";
  pubnub.addListener({
    message: function (msg) {
      console.log(msg);
      document.getElementById("message").innerHTML = msg.message;
    },
  });
```

```
});
pubnub.subscribe({
  channels: [channel],
});
</script>
```

Note: "channelName" The partner can use MerchantId to subscription, format is MerchantId+"_"+billNumber

```
final UserId userId = new UserId("7e576d4c-47df-4469-8455-36af98e8d8f3");
PNConfiguration pnConfiguration = new PNConfiguration(userId);
pnConfiguration.setSubscribeKey("sub-c-7750715e-b84d-4908-ae89-b3d840183cf9");

String channelName = "channelName ";
PubNub pubnub = new PubNub(pnConfiguration);
pubnub.addListener(new SubscribeCallback() {
    @Override
    public void message(@NotNull PubNub pubnub, @NotNull
PNMessageResult pnMessageResult) {
        System.out.println("Message: " +
pnMessageResult.getMessage());
    }
});

pubnub.subscribe()
    .channels(Arrays.asList(channelName))
    .execute();
```

Note: "channelName" The partner can use MerchantId to subscription, format is MerchantId+"_"+billNumber

4. Contact information

For more info, Please Contact my team as below:

- | | |
|---------------------------|--|
| 1. Mr. Sayher LORTOUA | sayher.lor@jdbbank.com.la
+856 20 29822100 |
| 2. Ms. Pouna Khamphengxay | Pouna.kha@jdbbank.com.la
+856 20 88704545 |
| 3. Mr. Santixay SICHANH | Santixay.sic@jdbbank.com.la
+856 20 88704545 |